

A vLLM serving simulator, and what it says about KV cache placement in DRAM

How a paged KV cache is laid out in physical memory, why it changes DRAM behaviour, what the simulator implements, how to measure it correctly, and what every experiment run so far actually showed.

Scope: **decode phase only**, LLaMA-2 7B, one transformer block. ONNXim + Ramulator2, HBM3 at 6.4 Gbps, 16 channels × 2 pseudochannels × 4 bank groups × 4 banks, 1 KB rows, 32 B requests, 819.2 GB/s peak. Default NPU is `_c128.json`, 4 cores × 128×128 weight-stationary systolic, fp16. Workload `az128`, 128 requests from the Azure LLM Inference Dataset, contexts 169–5,305 tokens, 200,064 tokens of cache, one decode step.

MEMORY

HBM3 · 6.4 Gbps · 819.2 GB/s
NPU

4 × 128×128 systolic, fp16

ORGANISATION

16 ch × 2 pch × 4 bg × 4 bank

WORKLOAD

az128 · 200,064 tokens

Summary

Two pieces of work. **A simulator**: a vLLM-style serving loop with paged KV allocation and per-step cache growth, on top of a cycle-level NPU and DRAM model, turning a real request trace into a cycle-stamped, per-stream DRAM address stream. **A layout study** run on it: where a paged KV cache should sit in physical memory, and what that is worth.

The simulator's enabling move is an approximation: **simulate a single transformer layer**. Consecutive layers touch disjoint rows, so no row buffer survives a layer boundary and there is no cross-layer locality to forfeit by stopping after one; a 32-layer step is 32 sequential repetitions of this one at different addresses. That takes a cycle-level decode step from ~4 days to ~3 hours, which is what makes the study feasible at all (§1).

Two findings, in descending order of how cheaply you can have them.

1. Page size is the largest single lever, and it is a serving-layer knob. Moving the paged KV block from 16 tokens to 64 takes DRAM row-buffer hit from **79.3% to 89.5%**, halves row activations, and runs **1.16× faster**, with no change to the layout code. At 64 tokens one page is exactly one DRAM row-stripe, so the allocator can no longer fragment a head's data across rows.

2. Putting the head index on the bank field is worth a further 4 points and 1.6× on activations. Aligning the KV base to a 512 KB boundary makes `bank = head` exactly: every head owns a bank no other head can address, so nothing evicts its rows. **93.5% hit, 3.2× fewer activations than the paged baseline.** The cost is bandwidth, a head confined to one bank has nowhere to read while that bank switches rows.

Both findings are decode-phase results and do not transfer to prefill. Measured on 8×256 -token prompts with no prior cache, the three placements land at 64.3%, 64.7% and 64.7% row hit on identical access counts, within 0.4 points of each other. During prefill the KV cache is being *written*, not read back, so there is no allocator-scattered history for a placement to arrange well; conflicts sit at 27% and are set by the weight and activation streams instead. Placement matters when the cache is large, old, and read repeatedly, which is the decode phase.

1. What this adds

ONNXim models a systolic NPU. Ramulator2 models DRAM. Neither models a serving system, and the combination could not produce a realistic KV access stream. Four things had to be built or fixed.

The KV path had to be made correct before it could be measured. Four defects in stock ONNXim meant there was effectively no KV traffic to study: every attention head read KV head 0 (`kv_head_idx = head_idx / _nkvh`), contexts past 2,730 re-read chunk 0 on every tile (tile-local `seq_idx`), the KV cache was never written at all (concat tiles marked `skip`), and value writes landed on the key tensor. Each produced a plausible-looking run. Details in Appendix C.

Paged allocation. ONNXim allocates KV as a contiguous tensor. A real server does not — it hands out fixed-size blocks from a pool, and a request's blocks end up wherever there was room. A block table was added (§3) so that logical block n is not physical block n . This is the difference between simulating a tensor and simulating a cache, and it is what makes allocator fragmentation a measurable effect rather than an assumption.

The approximation that makes this tractable: simulate a single transformer layer. The workloads here use `llama2-7b-1l`, a model declared with `num_hidden_layers: 1`, so one layer's DRAM traffic is what is simulated and reported. (The scheduler also supports declaring the full 32 and simulating one, in which case it scales the reported footprint by `_num_layers`; that path is not used by any result below.)

```
_num_sim_layers = _run_single_layer ? 1 : _num_layers;
```

The justification is about locality, not about the layers looking alike. **Consecutive layers touch disjoint rows:** layer $n+1$'s weights and its KV slice occupy different address regions from layer n 's, so layer $n+1$ can never hit a row buffer that layer n left open. There is no cross-layer row reuse for the controller to exploit, so truncating after one layer forfeits nothing it would have found. Every quantity this study measures, row-buffer hit, activations, bank occupancy, is determined *within* a layer.

The arithmetic this buys:

	PER DECODE STEP	10-STEP RUN
all 32 layers	~4 days	~40 days
1 layer, scaled	~3 hours	~30 hours

Without it, a single cycle-level decode step of a 32-layer model takes days, and multi-step is simply not reachable. Everything in Part III rests on this approximation.

Its cost is bounded and worth stating. The ~32 layer-boundary transitions per step are not modelled, against 6.4M KV accesses, and neither is any cross-layer tile overlap that a real run would produce and a single-layer run structurally cannot. The weight stream *is* present and interleaved with KV exactly as it would be, but only one layer's worth of it.

The footprint is also one layer's: 3.05 GB of KV at 200,064 tokens. A 32-layer deployment at that batch would need ~98 GB, well beyond the 16 GB modelled here. **This is the per-layer access pattern of a large batch, not a deployable memory configuration** — the batch size is chosen to make the KV stream dominant and realistic in shape, not to be servable on this part.

A fast path for speculative decoding. The cycle-level loop costs 35–45 minutes per verify step, which makes parameter sweeps impossible. An open-loop generator produces the same address stream in seconds, and the two agree request by request, so sweeps run on the generator and the cycle-level model confirms the operating points.

2. The serving loop

A decode step is not a kernel benchmark. Which addresses the KV cache touches depends on what a *paging allocator* did with earlier requests, so the access pattern cannot be derived from the model configuration: it has to be produced by running a serving loop. That is what this half of the work is.

The input is a request trace, one row per request:

```
time, prompt_length, target_length, cached_length
0, 1, 1, 237
0, 1, 1, 4089
```

`time` is arrival, `cached_length` is the KV already resident (so a decode-only workload can start mid-conversation), and `target_length` is where the request retires. `az128` is 128 rows of this, contexts 169–5,305, taken from the Azure LLM Inference Dataset.

Requests move `_request_queue` → `_active_requests` → `retired`, and **three schedulers differ in exactly one line, when a waiting request is allowed to join:**

SCHEDULER	ADMISSION CONDITION	BATCHING DISCIPLINE
<code>simple</code>	<code>_active_requests.empty()</code>	static: the batch must fully drain first
<code>iter_level</code>	<code>_active_requests.size() <= _max_batch_size</code>	continuous , joins a running batch, vLLM's iteration-level scheduling
<code>specdec</code>	as above, plus the draft/verify loop (§4)	speculative

Per step, every active request advances and its cache grows:

```

if (!gen_phase) { gen_phase = true; current_length += prompt_length + 1; } // prefill
else { current_length += 1; } // decode
for (i < _num_sim_layers) { key_cache[i]->resize_tensor({current_length, _cache_dim});
                          value_cache[i]->resize_tensor(...); }
if (current_length >= target_length) retire;

```

So the KV tensors genuinely grow step by step and requests retire at different times.

What the layout study actually exercises, stated plainly: every workload in Part III uses `target_length = 1` and the `simple` scheduler. They are *one* decode step over a batch of real contexts, not a multi-step run. The multi-step path is exercised by the speculative decoding loop (§4), which is where it is also validated. The `iter_level` scheduler is implemented and unused by any result reported here.

3. Paged KV allocation

The piece that makes the traces worth generating. A block table maps logical blocks to physical ones:

```

static std::map<uint32_t, std::vector<uint32_t>> cache; // one table per block-count,
                                                         // built once, reused
std::mt19937 rng(12345); // fixed seed: reproducible
if (!(mode && mode == "seq")) std::shuffle(table.begin(), table.end(), rng);

```

Three properties matter:

- **Shared.** One table per block-count, so every layer and every head of a run sees the *same* physical layout, as a real server would, since the allocator is global.
- **Scattered.** Logical block n is not physical block n , so a head's data is spread across the pool. This is what makes fragmentation a measured effect rather than an assumed one, and it is the reason page size turns out to be the largest single lever (§11): at 16 tokens a DRAM row-stripe holds four independently-placed pages.
- **Reproducible.** Fixed seed, so two runs differing only in layout are comparable.

What this is not, and what it costs. The scatter is a **uniform random permutation**, not a free list. A real vLLM allocator hands out blocks as sequences grow, reclaims them as sequences finish, and typically reuses the most recently freed first, so its physical pattern carries arrival and departure history and recently-freed blocks tend to return in runs. This model has no history: every logical block lands at an independent uniform-random physical index. It is therefore **maximal scatter**, and the two available settings bracket reality rather than reproduce it, `ONNXIM_KV_ALLOC=seq` giving zero scatter as the other end.

The consequence is specific and applies to one finding only. The page-size result works by making one page equal one row-stripe so the allocator *cannot* split a head's data

across rows; under an allocator that scatters less than uniformly there is less fragmentation to remove, so **79.3% → 89.5% is an upper bound on that benefit**. The head→bank result is not exposed this way: `bank = head` whatever physical block the allocator chose, so the block index only ever reaches the row field, and that finding holds under any allocation policy.

Layout is applied *on top* of the block table, in `Attention::kv_address()` : the physical block index chosen by the allocator is fed through one of three placement functions (Part II). Allocation policy and placement policy are therefore separable, which is what lets the study attribute effects to one or the other.

4. Speculative decoding

`scheduler: "specdec"` models the vLLM control flow exactly: draft $\times k$ at one query token per request, target verify at `q_len = k+1`, acceptance sampled from a geometric or replayed from a file, and the KV rollback that follows. Verify writes $k+1$ lookahead rows; the rows past the accepted count are stale and get overwritten by the next step, traffic with no analogue in plain decode.

It runs under the same single-layer approximation as §1, applied to both models (`_num_sim_layers` for the target, `_draft_sim_layers` for the draft). What is faithful is the *sequence of accesses* a speculative step produces, not the depth of the network producing them. **Acceptance is an input, not a result**: with no logits and one layer, whether a draft token would be accepted cannot be derived, so it is sampled from a geometric with `spec_alpha` or replayed from a file. The simulation is closed over KV bookkeeping, not over model output. A full 32-layer speculative run is out of reach for the same reason plain decode is: the cycle-level loop already costs 35–45 minutes per verify step at one layer.

Both vLLM scorers are modelled, and they differ by an order of magnitude in KV traffic:

SCORER	SHAPE	KV TRAFFIC
<code>mqa</code> (default)	one attention call, <code>q_len = k+1</code> , cache read once	1×
<code>expand</code>	batch expansion into $k+1$ sequences at contexts $L \dots L+k$	$k+1\times$

5. What it produces

One line per 32 B DRAM request, in arrival order at the controller:

```
cycle,channel,pseudochannel,bankgroup,bank,row,column,address,rw,core,operand
7,0,1,0,0,192,12,0x6004c00,R,0,102
```

`cycle` is DRAM cycles at 3.2 GHz ($\times 0.3125 = \text{ns}$), `address` is the global byte address, and `bank slot = pseudochannel + 2·bankgroup + 8·bank`.

The `operand` column is what makes the trace analysable. Each request carries the issuing instruction's operand id through `MemoryAccess` → `mem_fetch` → Ramulator's

`Request::source_id`, so **K, V, Q, weights, activations and outputs stay separable down to the row-buffer statistics** (Appendix B). Without it the streams cannot be told apart: activation tensors share the KV pool's address region and run at ~37% row hit, so any address-range split silently folds them into KV and understates its locality by several points.

Retained traces for the three main layouts are 122,117,889 requests each, identical bytes, ~0.77 GB compressed.

6. Why KV and not weights

A decode step runs the batch through the linear layers, then computes attention for each request against its own cache. The two streams behave completely differently.

The weight stream is fixed. The array is weight-stationary: it holds one 128×128 tile and streams every request's activation rows through it before replacing it. Batching therefore reuses *the loaded tile* across more rows, which raises arithmetic intensity inside the array. It cannot reduce the weight bytes crossing the DRAM boundary, because those are already at their floor: each weight is read exactly once per step at any batch size. Measured: **405 MB per transformer block, identical at batch 1, 8 and 128.** Weight addresses are fixed at load time and identical every step.

The KV stream scales with the batch. One token of one head is `head_dim × precision` = 256 B of keys and the same of values, so a step reads 16 KB per token across 32 heads. Weight traffic stays flat as the batch grows while KV grows with it, so KV's share rises with batch size. At 200,064 cached tokens that is **3.3 GB of KV against 405 MB of weights** in a single step, 89% of DRAM traffic. KV addresses are also produced by a *paging allocator*: a request's blocks are wherever the allocator had space, so the layout is not a function of the model configuration.

The address *set* can still be generated analytically, and `scripts/gen_trace.py` does exactly that as an independent check. What it cannot produce is the **order** those addresses reach the controller in, which is set by the tile schedule, four cores issuing concurrently, and the interconnect. Every row-buffer outcome depends on that order: the same addresses arriving in a different sequence give a different hit rate. That is what the cycle-level run supplies and a synthetic pattern cannot.

Within a page, though, the arrangement is a free choice, and that choice decides which addresses are adjacent, therefore the longest contiguous run any traversal can produce. Nothing downstream lengthens a run: a scheduler can reorder requests but cannot re-neighbour them, and a controller can only exploit locality placement already created.

Phases are separated in time. In the linear layers DRAM sees weights and activations finely interleaved (alternating at individual-request granularity, measured on channel 0 during `qkvgen`: runs of 1–3 same-class requests, occasional bursts of 64). During attention it sees keys and values only, one request at a time, that request's heads spread across the cores.

The separation is near-total. Bucketing an `az128` trace into 10,000-cycle windows: 1,965 of 2,322 windows carry KV traffic, 243 carry weight traffic, and **exactly one carries both**. The attention phase is **84.6% of the step's cycles**, and KV is 84.0% of its DRAM accesses.

7. What a row visit costs

A DRAM bank cannot read a cell directly. It must first *activate* a row — copy ~1 KB of cells into the sense amplifiers — costing `tRCD` (23 cycles here), during which that bank delivers nothing. Column reads then stream cheaply out of the open row. Before a different row can open, the current one is written back and the bitlines restored: `tRP`, another 23 cycles. **One row buffer per bank**; 32 banks per channel here.

A request that misses the open row fails one of two ways, and they do not cost the same:

OUTCOME	BANK STATE	COMMANDS	COST
hit	open on the wanted row	CAS	data only
miss	closed	ACT + CAS	~23 cycles, on an idle bank
conflict	open on a different row	PRE + ACT + CAS	~46 cycles, queue stalled behind it

The controller's precharge audit confirms the distinction is physical: across a full run **no precharge is ever left unfollowed by an activate** (`never followed 0.0%`), so a miss is not a wasted close: it is a close whose cost was successfully hidden off the critical path. A conflict is a close that was not.

The two metrics that matter

Reads per row visit = $1 / (\text{miss\%} + \text{conflict\%})$ on the KV stream. How much of a row visit's fixed cost is amortised. A 1 KB row holds 32 requests of 32 B, so **32 is a full row** and is the ceiling. A perfectly sequential walk gets $31/32 = 96.9\%$ **hit**, because the first touch of each row must miss.

Opens per row = $32 \times (\text{miss\%} + \text{conflict\%})$: the reciprocal view. How many activations one row of data costs, floor 1.0. Preferred when attributing causes, because unlike reads-per-visit it decomposes additively into the two things the controller reports:

```

opens/row = 32 × conflict% + 32 × miss%
              someone evicted   bank was CLOSED

block pg16    6.6 = 3.84 + 2.75
head-major    9.6 = 2.53 + 7.07
headbank pg16  2.6 = 1.57 + 0.99

```

Note headbank's miss term, 0.99 opens/row, is almost exactly the **first-touch floor**: a row holds 32 requests, so $1/32 = 3.125\%$ of accesses must miss simply because the row

has never been opened. Measured KV miss for headbank K=1 is 3.1%. It has essentially **zero avoidable misses**, every row it opens, it opens because it had to.

While-busy bandwidth = bytes delivered ÷ (cycles with ≥1 request outstanding × peak). Separates the memory system's efficiency from the NPU's ability to keep it fed.

Banks live — the variable a hit rate cannot see

A bank in a row switch delivers nothing, so whether the bus stays busy depends on how many *other* banks hold open rows with queued work. Measured directly from traces as the number of distinct banks a channel touches in a 2,000-cycle window (channel 0, KV only):

block-major	median	8 of 32	while-busy	68.7%
head-major	median	27 of 32	while-busy	98.5%
headbank K=1	median	4 of 32	while-busy	71.5%

This is the quantity that orders the placements by speed. Row hit rate does not.

8. The placement axis

Every KV placement here, including plain paged block-major, is the same flattening:

$$\text{addr}(\text{head}, p) = \underbrace{(p / U) \times (NH \times U)}_{\text{outer}} + \underbrace{\text{head} \times U}_{\text{head}} + \underbrace{(p \bmod U)}_{\text{inner}}$$

p = position in bytes within *that head's own* stream, NH = KV heads (32), and u = **bytes of one head kept contiguous** before the layout switches heads. Cut a head's stream into u -sized pieces; piece c goes to slot head of the c -th group of NH pieces. One parameter.

u is not a free-floating choice: it is read directly off the layout expression. The block-major branch (`Attention.cc`) strides heads by $B \times _dk$:

$$\text{off} = \text{phys} \times \text{blk} + \underbrace{\text{head} \times B \times _dk}_{\text{head stride = one page of one head}} + (\text{seq_idx} \% B) \times _dk + d;$$

SO $u = B \times _dk \times \text{precision}$. For this model $_dk = _dmodel / _nh = 4096/32 = 128$ and $\text{precision} = 2$, giving $u = \text{page size} \times 256 \text{ B}$:

U	PAGE	PLACEMENT	ROW-STRIPES PER HEAD
4 KB	16	block-major at vLLM's page size (baseline)	$\frac{1}{4}$ — four heads share a row
8 KB	32	—	$\frac{1}{2}$
16 KB	64	headbank K=1	1. one head per row
32 KB	128	headbank K=2	2
64 KB	256	≈ head-major	4
context	—	head-major	whole context

On the baseline. 16 tokens is vLLM's conventional page size, not a simulator default:

`ONNXIM_KV_BLOCK` unset makes `kv_block_tokens()` return 0, and `kv_address` then falls through to the flat `[head][seq][dk]` tensor: unpaged, and contiguous per head, so nearer head-major than block-major. Every run here pins `ONNXIM_KV_BLOCK=16` (`scripts/run_sweep.sh`), which is what makes 4 KB the baseline.

Head-major is the limit $U \rightarrow \infty : p < U$ always, the outer term vanishes, and the expression reduces to `head × context + p`, which puts the head index *above* the row field so a head's consecutive rows land in consecutive **banks** (it hops).

Where block-major meets headbank. At page 64 the block-major expression becomes `chunk = phys × 32 + head`, and `phys × 32 ≡ 0 (mod 32)`, so the bank field reduces to `head`: the same function headbank K=1 computes at page 64. The two differ only in that headbank pads the tensor base to 512 KB (§9.3).

Measured on the same build, the two issue an **identical** number of accesses (6,410,112 each): the same footprint, but do not reach the same hit rate: **89.5% against 93.5%**. They are the same mapping, not the same addresses. Block-major's tensor base is not stripe-aligned, so `bank = (base/16 KB + head) mod 32` and each head's 16 KB straddles two stripes.

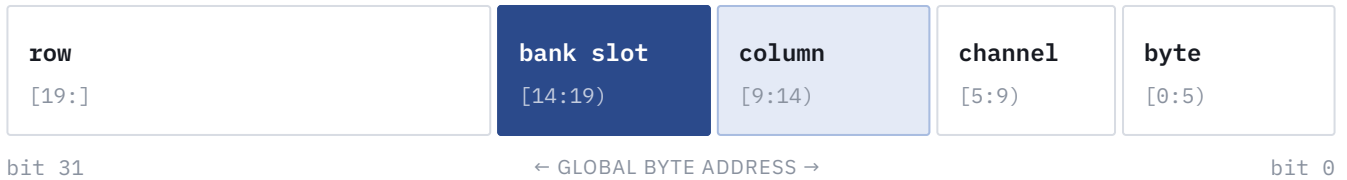
That alignment is worth +4.0 points of hit and 38% of the activations (673,062 → 416,657).

`U = B × head_dim × precision`, so the equivalence is model- and precision-specific: at int8 the same 16 KB is 128 tokens. **"One row-stripe per head" is the portable statement;** "page 64" is what it means for this model on this memory.

9. Headbank — the head index on the bank field

10.1 The address map

ONNXim hashes the channel out of the low bits and hands the compacted address to Ramulator, whose linear `RoBaRaCoCh` mapper slices from the bottom. In **global** address bits for this configuration:



row	increments every 512 KB = 2^{19} , after all 32 bank slots are used
bank slot	pseudochannel(1) + bank group(2) + bank(2) = 32 banks. This is the field headbank writes the head index into.
column	32 requests × 32 B = one 1 KB row per channel
channel	hashed away by ONNXim before Ramulator sees it, 16 channels
byte	offset within the 32 B request

The `RoBaRaCoCh` field layout for this configuration, verified independently against the trace decoder and Ramulator's mapper. A 16 KB contiguous run (2^{14}) fills exactly one row of one bank in every channel — a row-stripe.

Two constants follow:

- **16 KB = 2^{14} .** A 16 KB contiguous run scatters its 512 requests 32-to-a-channel, where they differ only in the column field, so one such run is **one row of one bank in every channel**. That is a *row-stripe*.
- **512 KB = 2^{19}** = 32 stripes, one per bank slot, after which the row index increments.

Substituting `U` = 16 KB, `NH` = 32 into §8 makes the formula a bit-packing:

$$\text{addr} = \underbrace{c \times 2^{19}}_{\text{row}} + \underbrace{h \times 2^{14}}_{\text{bank}} + \underbrace{\text{within}}_{\text{column}} \quad (\text{within} < 2^{14}, h < 32)$$

The head index is written into exactly the bits the mapper reads as a bank. **Nothing in Ramulator or the address mapper changes.**

Verified against the trace decoder and Ramulator independently: HBM3's `channel_width` defaults to 64 bits with prefetch 2, so both compute a 16 B transaction and agree on the field positions. (The trace's `column` values are always even because Ramulator counts columns in 16 B units while ONNXim issues 32 B requests.)

10.2 Worked example

Head 5, token 100, dim 0, 16-token pages → logical block 6, offset 4:

head 5 · token 100 · dim 0 · 16-token pages → logical block 6, offset 4

p	$(6 \times 16 + 4) \times 256 \text{ B}$	25,600
c	$25,600 / 16,384$	1 within = 9,216
addr	$1 \times 524,288 + 5 \times 16,384 + 9,216$	615,424 = 0x96400

$$0x96400 = 2^{19} + 2^{16} + 2^{14} + 2^{13} + 2^{10}$$

bits [19:]	1	row 1 = the chunk index
bits [14:19]	1,0,1,0,0 → 1+4	bank slot 5 = the head index
bits [9:14]	0,1,0,0,1 → 2+16	column 18

Token 164 → bank 5, row 2. Token 1,562 → bank 5, row 24. Head 6 at the same tokens → bank 6, same rows. **A head walks its own bank one row at a time; no other head can reach it.**

10.3 Two benefits, which fail independently

1. **The row is full of one head.** All 32 request slots of a row visit serve the head that opened it. A 4 KB unit fills 8 of 32, so $\frac{3}{4}$ of each activation's cost is data for heads not being read. Needs only $u \geq 16 \text{ KB}$, page 64 delivers it alone.
2. **The head owns the bank.** Nothing else can close head 5's row, so a visit ends when that head is done rather than when a neighbour arrives. Additionally needs the *alignment*: u exactly 2^{14} and the tensor base padded to 512 KB, so the head bits are not rotated by wherever the allocator placed the tensor.

Get u right and the base wrong and you keep (1) and blur (2): the head index still selects the stripe, but a base that is not stripe-aligned splits each head's 16 KB across two of them.

How much is each benefit worth? Head-major isolates (1) cleanly, because a head owns a contiguous region, so every row inside it holds exactly one head at *any* page size, while the bank is left to the allocator. Measured on the current build:

CONFIGURATION	ONE HEAD PER ROW	BANK = F(HEAD)	KV HIT
head-major pg16	yes	no	70.0%
head-major pg64	yes	no	70.7%
headbank K=1 pg64	yes	yes	93.5%

Benefit (1) alone is worth **0.7 points** across a 4× page range. Benefit (2) is worth **23 points**. The full row is a precondition for the isolation, not a substitute for it.

10.4 The K dial and the other knobs

$U = K \times 16 \text{ KB}$ gives each head K banks:

```
bank = (h·K + c mod K + tile.stride) mod NB
row  = (c / K) · S + h / (NB/K)          S = heads sharing a bank set = NH·K/NB
addr = row · (NB·CH) + bank · CH + (p mod CH)
```

- **K = 1**, one bank per head. Maximum isolation; a stream cannot overlap its own row switch, because its next row is in the same bank.
- **K = 2**. Head alternates two banks, so one delivers while the other precharges. Costs isolation: 32 heads over 16 bank-pairs means heads h and $h+16$ share a pair, separated in the row field.
- **V displacement**. $K(h)$ and $V(h)$ derive from the same offset and both bases are 512 KB-aligned, so **they land in the same bank**, and the attention tile reads them back to back. `ONNXIM_KV_V_BANK_OFFSET=16` moves V to bank $h+16$.
- **Tile stride**. For long contexts split into several M-tiles, offsets tile m by $m \cdot \text{stride}$ banks. Keyed on `tile->M`. Measured neutral (Appendix D).

10.5 Porting to another DRAM organisation

The two geometry constants are parameters, not literals:

```
ONNXIM_KV_BANKCHUNK  bytes per row-stripe = row size × channels      (default 16384)
ONNXIM_KV_NBANKS     bank slots per channel = pch × bankgroups × banks (default 32)
```

To port: find where the mapper places the bank field in the *global* address, set `BANKCHUNK` to $2^{(\text{low bit of that field})}$ and `NBANKS` to the field's width. Empirically verifiable by fitting the trace's decoded `bank / row` columns against its `address` column, which is how the constants above were confirmed.

10. The placement comparison

PLACEMENT	KV HIT	MISS	CONFL	RD/VISIT	KV ACT	CYCLES	WHILE-BUSY	VS BASELINE
block-major pg16 (baseline)	79.3%	8.8%	11.9%	4.8	1,326,887	24.19M	68.7%	—
block-major pg64	89.5%	4.5%	6.0%	9.5	673,062	20.84M	80.8%	1.16×
head-major	69.9%	22.3%	7.9%	3.3	1,935,917	17.73M	98.3%	1.36×
headbank K=1 pg16	91.9%	3.1%	4.9%	12.5	512,809	23.21M	71.5%	1.04×
headbank K=1 pg64	93.5%	3.4%	3.1%	15.4	416,657	21.46M	77.9%	1.13×
headbank K=2 pg64	85.0%	11.9%	3.1%	6.7	961,517	18.42M	92.3%	1.31×
headbank K=1 pg128	93.5%	3.3%	3.2%	15.4	416,657	21.46M	77.6%	1.12×
K=1 pg64 + V16	90.3%	6.5%	3.2%	10.3	621,781	20.39M	82.5%	1.18×
K=1 pg64 + V16 + RR	84.5%	10.9%	4.6%	6.5	993,567	21.34M	87.3%	1.13×
headbank K=2	83.2%	11.9%	4.9%	6.0	1,070,489	18.88M	90.2%	1.28×
K=2 + V16 + pg64	82.2%	14.6%	3.2%	5.6	1,141,000	18.12M	94.4%	1.33×

Reading it. Head-major is fastest with the *worst* locality and the *most* activations. Its advantage is bank-level parallelism, not row locality.

Block-major's higher hit rate is *shared*, and the sharing is structural. Its page is `[block][head][token][dk]` = $32 \times 16 \times 128 \times 2 = \mathbf{128\ KB}$, in which one head's fragment is 4 KB, so four heads occupy each 16 KB row-stripe, and a 128 KB page spans exactly 8 of the 32 bank slots. Both numbers are measured directly off the trace:

distinct 4 KB head-fragments per bank slot	4.00	(= 16 KB / 4 KB)
distinct bank slots per 128 KB page	7.57	(= 128 KB / 16 KB, partial pages sampled)

So when the cores work adjacent heads they land in one row and each finds it already open — a real hit, but one no single stream earns; four streams take turns filling a row's 32 slots. The same mapping caps bandwidth: **8 banks live is the page's entire bank footprint**, not a scheduling artifact, so no amount of extra concurrency can exceed it. Head-major, hopping a bank every 16 KB, reaches all 32 (27 live per window).

The two layouts fail differently, and that is what inverts the ranking. Of block-major's 20.6% non-hits, 58% are conflicts, four heads in one bank evicting each other. Of head-major's 30.0%, only 26% are: its failures are cold opens on idle banks. Weighting by §7's costs:

block	$8.6\% \times 23 + 12.0\% \times 46$	=	750	stall-cycle-units per 100 accesses	
head	$22.1\% \times 23 + 7.9\% \times 46$	=	872		(+16%)

Head-major carries *more* nominal stall and still finishes 27% sooner, because its stalls overlap across 27 banks while block-major's are exposed on 8. A hit rate counts failures; it cannot see how many of them are concurrent. That gap is the subject of §13.

Three operating points worth knowing:

- **K=1 pg64**, best locality and fewest activations. 3.2× fewer KV activations than the baseline for identical bytes, and 1.12× faster. Choose it when DRAM activation energy is the objective. Page 128 is identical: at page 64 one page *is* one row-stripe, fragmentation is eliminated, and larger pages have nothing left to fix.
- **K=1 pg64 + V16**, best joint point: 90.3% hit *and* 82.5% bandwidth, 1.18× faster.
- **K=2 + V16 + pg64**, best time among isolated placements, 94.4% bandwidth, 1.33×.

11. Page size

For **headbank**, page size is the strongest single lever (Appendix D): 91.9 → 93.5% hit, 12.5 → 15.4 reads/visit, −19% activations, −7% runtime, and it saturates at 64. Cause: a 16 KB stripe spans four 16-token pages, which a fragmenting allocator can scatter across rows; at page 64 one page *is* one stripe and it cannot.

For **head-major**, page size is inert: **70.0% at page 16 against 70.7% at page 64** on the current build, with activations down 2.3% and runtime unchanged within noise (17,683,076 → 17,687,655 cycles). The reason is that head-major never depended on the page for row purity, a head owns a contiguous region, so every row in it holds that head alone whatever the page size. What the page cannot fix is the bank, which stays allocator-chosen.

For **block-major**, page size *is* the axis of §8: it moves the placement along it, and this is the most practically useful result here: **page 16 → 64 alone gives 79.4% → 89.7% hit, halves KV activations, and runs 1.16× faster, with no layout code at all.** At page 64 the block-major expression already puts the head index on the bank field; what it still lacks is the stripe-aligned base, worth a further 3.8 points (§8). For anyone running a paged KV cache, changing the page size is the cheapest available move and captures most of the benefit.

12. Core count

CONFIGURATION	BLOCK	HEAD-MAJOR	K=1	K=2
1 core × 128×128, 8 req	76.7%	78.4%	92.0%	93.8%
4 × 128×128, az128	79.4%	70.0%	91.9%	83.2%
16 × 32×32, az128	68.8%	74.1%	66.3%	73.3%
16 × 128×128, az128	—	—	66.1%	—

At 16 cores the placement effect **collapses**: hit lands in a 66–74% band and runtime in a 23.9–25.3M band regardless of placement, while bandwidth is uniformly 91–98%. K=1 goes from best locality to worst.

What does *not* explain it: conflicts stay at 4.9–5.0%, identical to four cores, so isolation has not broken and heads are not evicting one another: the losses are all misses. Nor is it array width: the $16 \times 128 \times 128$ control behaves the same as $16 \times 32 \times 32$. **What collapses is run length**, $15.4 \rightarrow 2.9$ reads per visit. The mechanism is unresolved.

Note $16 \times 32 \times 32$ also uses a different DRAM config (32 GB); the $16 \times 128 \times 128$ run is the clean control. The two NPU configs are *iso-array-floor* ($\text{core_width} \times \text{num_cores} = 512$ both ways) but not *iso-compute*: $4 \times 128 \times 128$ has 4× the PEs.

Practical reading: this placement work pays on wide-array machines and not on many-narrow-core ones.

13. The bandwidth/locality trade

Every mechanism tried to raise bank-level parallelism bought bandwidth and paid in run length. Measured bank counts (2,000-cycle windows, channel 0, KV only):

CONFIG	BANKS LIVE	BANKS PER CORE	RD/VISIT	WHILE-BUSY
headbank K=1	2.8	1.03	12.5	71.5%
block-major	6.6	4.93	4.9	68.7%
head-major	28.4	19.99	3.3	98.5%

The mechanism is per-core bank count, and it is a property of the address map.

Under headbank $\text{bank} = \text{head}$, and a core processes one head at a time, so a core is confined to exactly one bank — 1.03 measured. Its next chunk is in *that same bank* at the next row, so every row boundary is a mandatory ~46-cycle close-and-reopen with nothing else to issue. Head-major's stream hops a bank every 16 KB, so one core sweeps 20 banks and their row switches interleave. Note the same ~2.9 cores are active in all three: the difference is entirely banks per core, not concurrency.

The saturation condition is $\text{banks_per_pseudochannel} \times \text{duty} \geq 1$:

headbank K=1	$1.4 \times 0.64 = 0.90$	below 1.0, bus starves	$\rightarrow 77.4\%$
head-major	$14.2 \times 0.31 = 4.46$	4.5x oversupplied	$\rightarrow 98.5\%$

Head-major's individual banks are **half as efficient** (31% duty vs 64%); it wins by having ten times as many.

The trade is not a conserved quantity. The product $\text{banks} \times \text{rd/visit}$ is 35, 32 and 94 across the three layouts: bandwidth is not bought from a fixed budget of outstanding requests. It is governed by the coverage condition above and nothing simpler.

Whether the trade is fundamental is open. K=2 gives each head two banks so one delivers while the other switches, and it works: while-busy 77.9% → 92.3%, 1.16x faster. But it costs 8.5 points of hit, and that cost has no identified cause. Each `(bank, row)` still holds one 16 KB chunk of one head, every chunk is still read in full, and conflicts are **identical at 3.1%**, so isolation is perfectly intact. The entire loss is *misses* — banks found closed. Ruled out by measurement: refresh (Appendix C bug 10; enabling it moves hit by 0.0 points), inter-head eviction (conflicts flat), and auto-precharge (no RDA/WRA issued).

If those misses turn out to be the controller discarding rows the stream immediately wants back, then "you cannot have both" is a defect rather than a law, and K=2 would sit at ~93% hit with ~92% bandwidth. A close-by-cause audit — attributing every KV miss to first touch, demand precharge, or refresh, and flagging same-row-came-back — is the measurement that decides it.

14. Two controller defects, both null

Two genuine controller bugs were found and fixed. Neither changes any result, which is worth recording precisely because the instinct is to assume they would.

Refresh was never issued (Appendix C). With it fixed and firing correctly (1,716 sweeps at `nREFI` = 12,500, `nRFC` = 1,122):

LAYOUT	HIT OFF → ON	CYCLES OFF → ON
block-major pg16	79.4% → 79.3%	24.13M → 24.19M
block-major pg64	89.7% → 89.5%	20.81M → 20.84M
head-major	70.0% → 69.9%	17.68M → 17.73M
headbank K=1 pg64	93.5% → 93.5%	21.55M → 21.46M

Largest change anywhere: **0.2 points of hit, 0.25% of runtime.** A `nRFC` / `nREFI` duty of 9% costs nothing here for two reasons: only a few banks hold open rows at any instant (2.8 under headbank), so a global precharge destroys almost nothing; and the 1,122-cycle blackout drains a 941-deep request queue rather than stalling the NPU. Every comparison in this document is unaffected.

Ramulator's `FRFCFS` is not FR-FCFS. It ranks ready-over-unready then falls back to arrival order, but a row hit and a row miss are both "ready," so the tie goes to age and an older miss beats a younger hit. There is no row-hit tier. Adding one (`RAMULATOR_SCHED_ROW_HIT=1`):

LAYOUT	CONFLICTS	HIT → WITH TIER	GAIN	RUNTIME
headbank K=1 pg64	3.1%	93.5 → 93.6%	+0.1	-0.5%
headbank K=2 pg64	3.1%	85.0 → 85.8%	+0.8	-0.4%
block-major pg64	6.0%	89.5 → 89.9%	+0.4	-0.3%
head-major	7.9%	69.9 → 71.0%	+1.1	-1.6%
block-major pg16	11.9%	79.3 → 79.8%	+0.5	-1.2%

≤1.0 point on every layout. A predicted correlation between conflict rate and gain does not hold: block-major has the most conflicts and recovers least. The scheduler is not the lever at any conflict rate, and where placement already keeps rows alive it contributes exactly nothing.

Why a scheduler cannot substitute for placement. It is not a question of window depth: the controller reorders across ~941 queued requests, which is ample. The limit is that a scheduler reorders requests but cannot *re-neighbour* them. Placement decides which addresses share a row, and no ordering changes that. Under block-major a row holds four different heads' 4 KB fragments, so its 32 request slots can never serve one head however they are sequenced; the row-hit tier recovers 0.5 points there, against 79.3% hit and 11.9% conflicts, which is the most locality of any layout left on the table. If reordering were the constraint, the gain would scale with how much was available to recover. It does not.

That is a claim about *creating* locality. Preserving it is a separate matter, and there the controller is currently failing at something it could do (§15).

15. What closes a bank: the stranded-request audit

Three separate observations shared one fingerprint — K=1 → K=2, 4 → 16 cores, and headbank's residual opens/row. In each, **conflicts are unchanged** (isolation never breaks) and the entire loss is *misses*, i.e. banks found with no row open. Refresh is excluded (§14) and eviction is excluded by the flat conflict rate. So something else closes them.

The controller was instrumented to attribute every KV miss to what closed the bank, recording at each `PRE` which bank, which row was in the buffer, at what cycle, and the operand responsible. Result:

	KV MISSES	CLOSED BY DEMAND <code>PRE</code>	SAME ROW CAME BACK	MEAN GAP
headbank K=1 pg64	216,705	207,852 (95.9%)	207,851 (100%)	24 cyc
headbank K=2 pg64	765,680	750,192 (98.0%)	750,188 (100%)	26 cyc

The controller precharges a row and then wants that exact row back ~25 cycles later — about `tRP`. Each instance costs a `PRE + ACT` round trip, 46 cycles, to restore data that was sitting in the sense amplifiers when it was discarded. K=2 does this 750,188 times.

The cause is in the scheduler, and it is the same defect as §14: with no row-hit tier, `check_ready()` returns true for a request needing `PRE` *and* for one able to read the open row, so the tie falls through to arrival order. An older request wanting a different row outranks younger requests that could read right now, and serving it throws their row away.

The row-hit tier does not fix it, because it sits *below* the readiness test: when a pending hit is momentarily blocked by column timing it fails `check_ready()`, tier 1 picks the competing `PRE`, and the row-hit tier is never consulted. Measured: it prevents 2.6% of K=1's strandings and 7.3% of K=2's, which is exactly the ≤ 1 -point gain in §14.

Switch duration does not affect bandwidth. The precharge-to-activate gap runs 48–95 cycles against a 23-cycle floor, which looks like a defect and is not:

LAYOUT	PRE→ACT GAP	BANKS LIVE	WHILE-BUSY
head-major	94.5 cyc (worst)	28.4	98.3% (best)
headbank K=1	72.2	2.8	77.9%
block-major pg16	48.0	6.6	68.7%

Gap and bandwidth are uncorrelated, if anything inverted — more banks means the scheduler always has work elsewhere, so any one bank's reactivation waits longer, and it does not matter because others are delivering. This also explains `RAMULATOR_SCHED_BANKPREP` (Appendix D), which closed the gap to 28.5 cycles and ran 3.2% *slower*: it optimised a quantity that does not affect throughput. **Switch duration is irrelevant; switch count is not.**

Why this matters for the trade in §13: reads-per-visit appears in both metrics —

```
hit rate  opens/row = 32 / reads_per_visit
coverage  turn      = reads_per_visit x nBL      must exceed tRP+tRCD = 46 cycles
```

K=2 sits at 6.7 reads/visit against a natural ~16. At 16 its turn is 64 cycles, past the 46-cycle threshold where one partner bank suffices. So eliminating the strandings would raise locality *and* bandwidth from a single cause. A guard that refuses to precharge a bank holding queued hits — above the readiness test, unlike the row-hit tier — is implemented (`RAMULATOR_SCHED_KEEPOPEN`) and not yet measured.

16. Across the stack

The measurements place a value on each layer's decisions, and they are not evenly distributed:

LAYER	DECISION	MEASURED WORTH
Serving (vLLM)	page size 16 → 64	+10.2 pts hit, 2× fewer activations
Tensor layout	block / head / headbank	up to +23 pts
Address map	stripe-aligned base	+4.0 pts
Memory controller	row-hit tier, bank-prep, refresh	≤1.1 pts
DRAM device	<code>tRP</code> , <code>tRCD</code> , <code>nBL</code>	sets the ceiling: 32 reads/row, 46-cycle switch

Locality is created at the top of the stack and can only be preserved or squandered below it. That is not a slogan but the measurement: the scheduler is worth ≤ 1.1 points on *every* layout including the worst, so no controller recovers what placement did not create. Meanwhile a serving-layer configuration change that nobody thinks of as a memory decision is worth ten points.

The corollary is the more useful direction: **good placement is what makes downstream problems visible.** The 750,188 stranded requests of §15 were always present: they only became measurable once isolation removed the eviction noise, because conflicts stopped moving and every remaining loss had to be explained by something else. Bad placement hides controller defects behind its own noise.

Work that follows from this:

1. **Concurrent heads per core.** Under headbank the 32 heads already occupy 32 distinct banks, but only ~ 3 are ever active, so 2.8 of 32 banks are live. Every mechanism tried so far gave *one head more banks* and traded locality away. Running several heads per core concurrently costs no isolation by construction: they are already in separate banks and cannot evict one another. Untested, and the only remaining route to high locality and high bandwidth at once.
2. **The stranded-request guard** (§15), which would raise reads-per-visit and therefore both metrics from one cause.
3. **Half of every row is unaddressed:** the device declares 2 KB rows and the mapping reaches 1 KB. If unintentional, the ceiling on reads-per-visit is 64 rather than 32.

17. Speculative decoding

The specdec path is built and validated (§4) but the layout study has barely touched it. What is known: on a speculative workload the layout effect **disappears**: head-major and headbank K=2 land within 0.6%, because KV is only ~7% of that workload's DRAM traffic, the draft model's weights dominating instead.

That makes it interesting rather than closed. Speculative decoding changes the KV access pattern in ways plain decode does not: verify reads a $k+1$ *token* query against one cache pass, lookahead rows are written and then overwritten as stale, and the draft and target caches interleave two different stream populations over the same banks. The write-after-write traffic on rolled-back rows has no analogue in plain decode. None of that has been measured against placement, and the scorer choice (single-call MQA vs batch expansion) moves KV traffic by an order of magnitude, so the layout sensitivity may return entirely at the other operating point.

18. Open questions

1. **Behaviour under a hashing address mapper.** The placement assumes the bank index is a contiguous address field. A controller that folds row bits into it would scramble the head→bank assignment. No validated hashing implementation is available here, so all runtime ratios are conditional on a linear mapper.
2. **Concurrent heads per core: the untested lever.** Under K=1 the 32 heads already occupy 32 distinct banks, but only ~3 are ever active, so 2.8 of 32 banks are live. Every mechanism tried so far gave *one head more banks* (K=2, V16) or interleaved tiles sharing a bank (RR), and all traded locality away. Running several heads per core concurrently costs no isolation by construction: the heads are already in different banks and cannot evict one another. This has never been tested and is the only remaining route to high locality and high bandwidth simultaneously.
3. **GPU serving concurrency.** This simulator runs attention one request at a time, so at four cores the concurrent streams are twelve heads of *one* request. A paged-attention kernel spreads many requests across SMs, giving a different stream population over the same banks. Check before applying any of this to a GPU.

A. Knobs

KNOB	FILE	EFFECT
ONNXIM_KV_LAYOUT	Attention.cc	block (default) / head / headbank
ONNXIM_KV_BANKS_PER_HEAD	Attention.cc	the K dial, default 1
ONNXIM_KV_V_BANK_OFFSET	Attention.cc	shift V's base by N bank slots
ONNXIM_KV_TILE_BANK_STRIDE	Attention.cc	offset M-tile m by m.stride banks
ONNXIM_KV_REQ_ROTATE	Attention.cc	per-request bank rotation
ONNXIM_KV_BANKCHUNK / _NBANKS	Attention.cc	geometry, see §9.5
ONNXIM_KV_BLOCK	Attention.cc	page size in tokens
ONNXIM_ACT_LANE	Attention.cc	confine activation traffic to one bank slot
ONNXIM_WEIGHT_TILEBANK	Operation.cc	same isolation for the weight stream
ONNXIM_ICNT_PORT_BY_CHANNEL	Simulator.cc	inject on the port serving the request's channel
ONNXIM_RR_TILE_ISSUE	Core.cc	per-tile request sub-queues, drained round-robin
ONNXIM_TILE_LOG	Core.cc	log every tile dispatch (core, layer, N, M)
RAMULATOR_SCHED_BANKPREP	generic_dram_controller.cpp	issue PRE/ACT ahead of ready row hits
RAMULATOR_SCHED_ROWBIT / _STARVE_CAP	generic_scheduler.cpp	the row-hit tier stock FRFCFS lacks, and its fairness bound

All default to stock behaviour when unset; an unmodified run reproduces plain paged block-major exactly.

`ONNXIM_ICNT_PORT_BY_CHANNEL=1` **should be considered the correct default**. Stock fans a core's single request FIFO across 16 injection ports, and the interconnect drains a channel's inputs round-robin across those ports, so a core's requests arrive at one channel locally reordered. A bank-hopping stream doesn't notice; a bank-confined one gets rows r and $r+1$ interleaved and the controller ping-pongs them. One FIFO per (core, channel) — which is what a crossbar gives a single flow — cut row conflicts **35–45% on every placement**. All results below use it.

B. Measuring it correctly

B.1 Separate the streams at the controller

Row-buffer statistics are meaningless if streams are mixed, and **they cannot be separated by address range**: activation tensors share the KV pool's address region and run at ~37% row hit, so averaging them into KV understates it by several points.

Each request now carries the issuing instruction's `operand_id` through `MemoryAccess.operand` → `mem_fetch.operand` → Ramulator's `Request::source_id` (unused otherwise). The controller prints three lines:

```
ROWSPLIT weights  acc ... hit ...% miss ...% confl ...%
ROWSPLIT kv+act   acc ... hit ...% miss ...% confl ...%   ← KV only: operand 101/102
ROWSPLIT act      acc ... hit ...% miss ...% confl ...%
```

Operand codes: **100 = Q, 101 = K (or a GEMM weight when below the weight limit), 102 = V, ≥200 = outputs, 0 = KV writes**. The same tag is written into the trace CSV, so offline analysis needs no threshold either.

B.2 Deriving the numbers

```
KV reads per visit = 1 / (KV_miss% + KV_conflict%)
KV activations     = KV_accesses × (KV_miss% + KV_conflict%)
avg BW             = (reads_served + writes) × 32 B × n_channels / (dram_cycles / 3.2e9) / 819.2e9
while-busy BW      = avg BW / channel_busy_fraction
banks live         = distinct (pch,bg,bank) per channel per 2,000-cycle trace window
```

`reads/ACT` taken from the `CTRL cmd mix` line is an **all-stream aggregate** and is diluted by the weight stream, do not quote it as a KV figure. KV-specific is 12.5 where the aggregate says 8.4.

B.3 Two pitfalls that produced plausible wrong numbers

1. **The address-threshold stream split** (fixed, see 6.1). Reported K=1 at 87.5% when the truth was 91.9%.
2. **Trace scripts must use the full `RAMULATOR_WEIGHT_LIMIT` (500,000,000)**. The CSV's `address` column is the *global* address; only the controller sees the ÷16 compacted form. Applying the compacted threshold to the CSV classes 97% of the *weight* stream as KV. Hours of "KV stream" analysis were the weight stream wearing a KV label, and three placement knobs were designed against it. **Use the `operand` column**.

B.4 Validation

The simulator is deterministic, identical configs give bit-identical cycle counts, so every delta reported is real and the only question is whether it is large. The independent

arrival of page-64 block-major and headbank K=1 at the same runtime (0.04% apart) from two unrelated derivations is a useful cross-check on the address generation.

C. Reproduction

Common environment (all runs below):

```
E='ONNXIM_WEIGHT_SWIZZLE=1 RAMULATOR_REQBUF=1024 RAMULATOR_WEIGHT_LIMIT=500000000
ONNXIM_KV_WRITES=1 ONNXIM_ICNT_PORT_BY_CHANNEL=1'
```

```
# baseline: paged block-major, 16-token pages
env $E ONNXIM_KV_BLOCK=16 ONNXIM_KV_LAYOUT=block \
./bin/Simulator --config ../configs/_c128.json \
--models_list ../example/az128.json --mode language --trace_file az128.csv

# head-major
env $E ONNXIM_KV_BLOCK=16 ONNXIM_KV_LAYOUT=head ...

# headbank K=1 at page 64 – best locality, fewest activations
env $E ONNXIM_KV_BLOCK=64 ONNXIM_KV_LAYOUT=headbank ONNXIM_KV_BANKS_PER_HEAD=1 ...

# best joint point
env $E ONNXIM_KV_BLOCK=64 ONNXIM_KV_LAYOUT=headbank ONNXIM_KV_BANKS_PER_HEAD=1 \
ONNXIM_KV_V_BANK_OFFSET=16 ...

# add a cycle-stamped trace to any run
ONNXIM_DRAM_TRACE=/path/out.csv
```

Trace format — one line per 32 B DRAM request, arrival order at the controller:

```
cycle,channel,pseudochannel,bankgroup,bank,row,column,address,rw,core,operand
7,0,1,0,0,192,12,0x6004c00,R,0,102
```

`cycle` = DRAM cycles at 3.2 GHz ($\times 0.3125$ = ns), `address` = global byte address, bank slot = `pseudochannel + 2·bankgroup + 8·bank`.

Retained traces: `out/traces_layout/az128_{head,headbank_k1,block}.csv.gz` — 122,117,889 requests each, identical bytes, same settings, ~0.77 GB compressed.

D. Negative results — measured, do not re-run

ATTEMPT	RESULT
RoCoBaCh mapper (bank bits below column)	2× slower both layouts; forces ~32 activates before a row is consumed
tile_depth 6	±1%; adds queued tiles, not streaming ones
ONNXIM_PAR_STRATEGY=request	KV hit +10 pts, channel busy 59%, 1.6× slower
V-bank shift alone (pg16)	91.9 → 89.0% hit, 71.5 → 76.2% BW – a trade, not a win
ONNXIM_ACT_LANE (activation isolation)	neutral
ONNXIM_KV_REQ_ROTATE	neutral
ONNXIM_KV_TILE_BANK_STRIDE	neutral once request ordering was fixed
RAMULATOR_SCHED_BANKPREP	PRE→ACT gap 62 → 28.5 cycles, runtime 127 cycles in 7.4M at 1 core, 3.2% worse at 4 cores
ONNXIM_WEIGHT_TILEBANK (weights)	73.9 → 73.1% hit, runtime -0.2%
ONNXIM_RR_TILE_ISSUE (round-robin)	bandwidth +6-10 pts, run length -45 to -58%, runtime flat or worse
RAMULATOR_SCHED_ROW_HIT (the row-hit tier FRFCFS lacks)	≤1.0 pt of hit on all four layouts; +0.0 on headbank (§15)
enabling refresh (bug 10 fixed)	≤0.2 pts of hit, ≤0.25% runtime, on all four layouts (§15)
page size for head-major	inert, +2.5 pts over a 16× range
speculative decoding	layout-insensitive; head vs K=2 within 0.6% (KV is 7% of that workload)

On bank-prep specifically: the tier worked exactly as designed: the gap fell to the timing floor, and changed nothing, because when every queued request targets a bank mid-row-switch there is nothing left for a scheduler to reorder. **The controller cannot pay for isolation.**